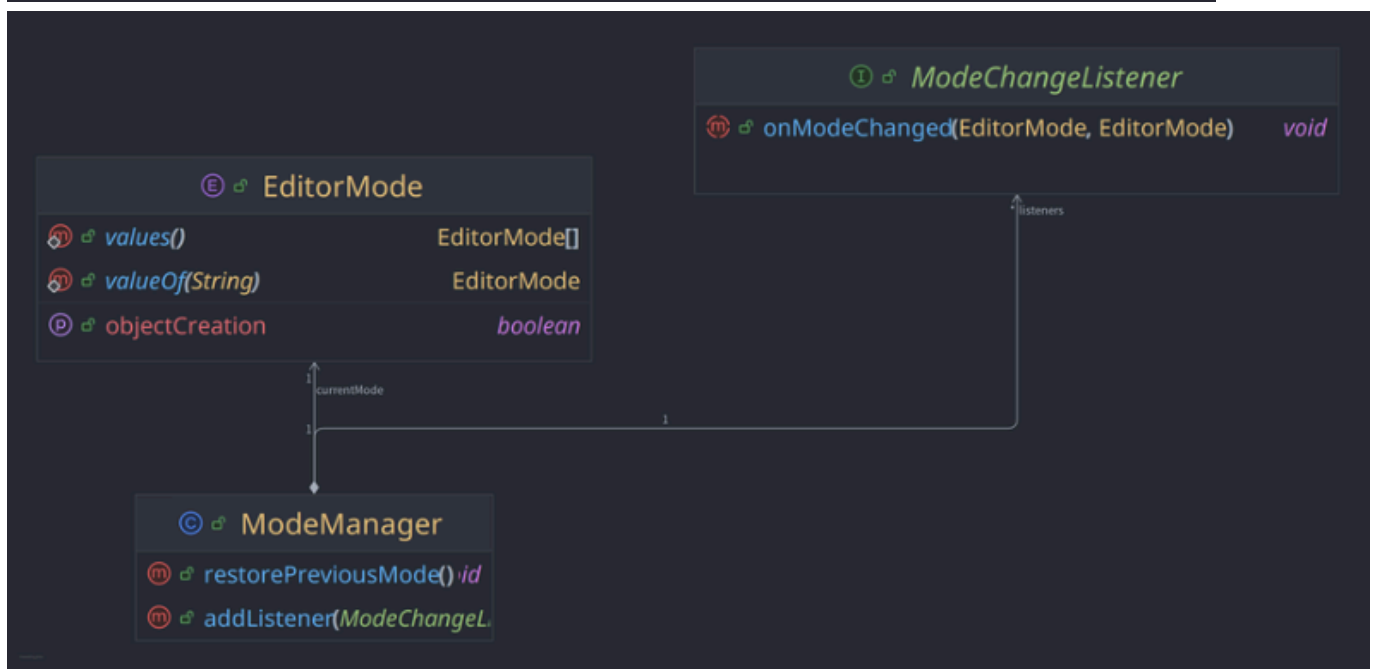
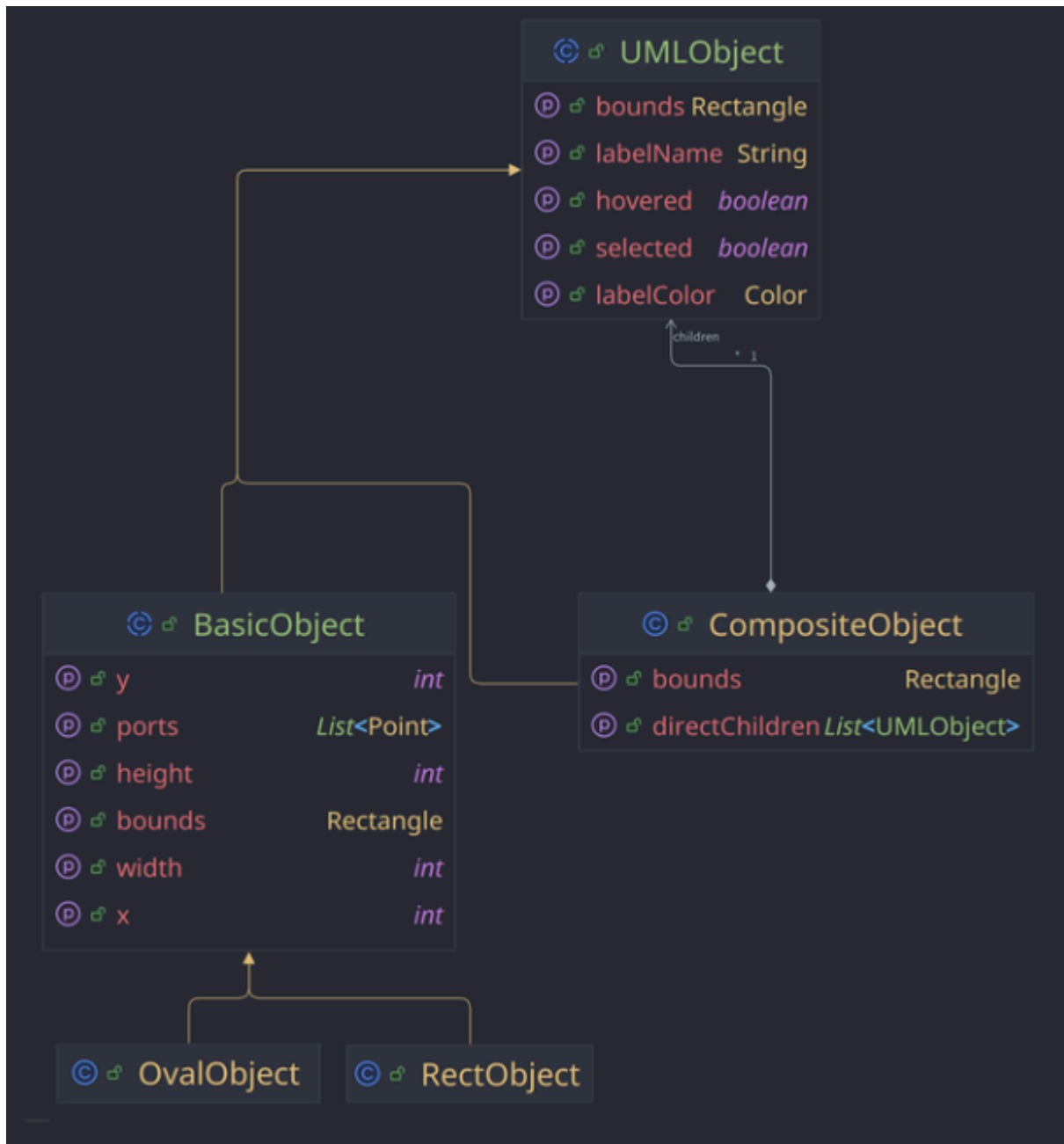
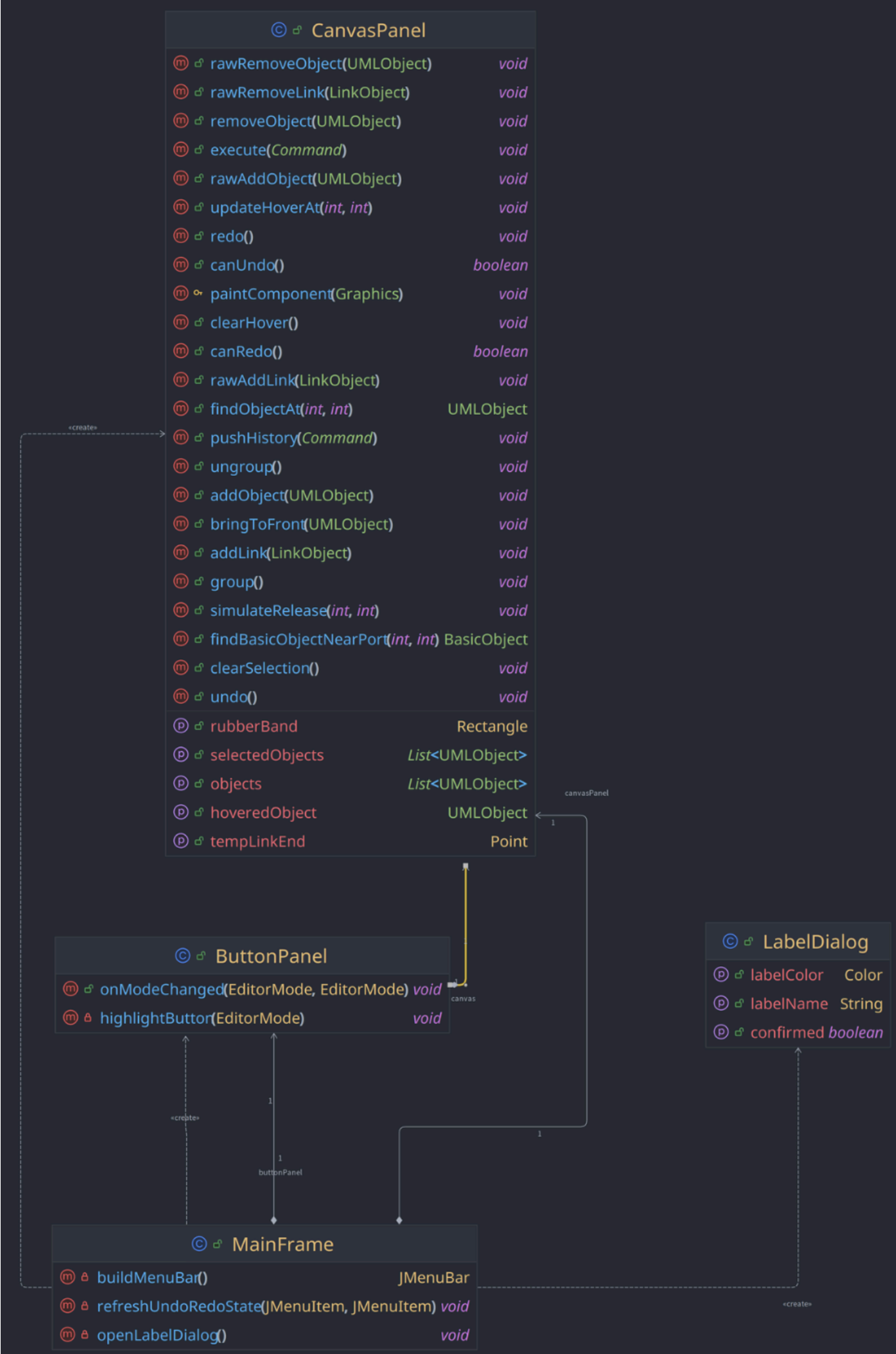


— 、Class diagram with methods and key attributes







二、每個方法的概略行為

CanvasPanel（畫布主控）

方法	行為
<code>execute(cmd)</code>	呼叫 <code>cmd.redo()</code> 套用變更，再把 <code>cmd</code> 推入 <code>undoStack</code> ；適用「一次性建立」操作
<code>pushHistory(cmd)</code>	只推入 <code>undoStack</code> ，不重複執行 <code>redo</code> ；適用已即時更新的 <code>Move/Resize</code>
<code>rawAddObject(obj)</code>	直接把 <code>UMLObject</code> 加入 <code>objects</code> 列表，不走 <code>Command</code> ；供 <code>Command.redo/undo</code> 呼叫
<code>rawRemoveObject(obj)</code>	從 <code>objects</code> 列表移除物件；供 <code>Command.undo</code> 呼叫
<code>rawAddLink(link)</code>	把 <code>LinkObject</code> 加入 <code>links</code> 列表
<code>rawRemoveLink(link)</code>	從 <code>links</code> 列表移除 <code>LinkObject</code>
<code>findObjectAt(x,y)</code>	依 <code>depth</code> 排序後，回傳第一個 <code>contains(x,y)==true</code> 的物件
<code>findBasicObjectNearPort(x,y)</code>	掃描所有 <code>BasicObject</code> ，回傳距離最近的 <code>port</code> 在範圍內的物件
<code>group()</code>	確認 ≥ 2 個物件被選取後，建立 <code>GroupCommand</code> 並 <code>execute</code>
<code>ungroup()</code>	確認唯一選取物件是 <code>CompositeObject</code> ，建立 <code>UngroupCommand</code> 並 <code>execute</code>
<code>simulateRelease(x,y)</code>	由 <code>ButtonDragGlassPane</code> 呼叫，模擬滑鼠在畫布上放開，觸發 <code>CreateObjectStrategy.onReleased</code>

SelectStrategy（選取/移動/縮放策略）

方法	行為
<code>onMoved(e, canvas)</code>	hover 偵測：找物件 → <code>setHovered(true)</code> 顯示 <code>ports</code> ，離開後 <code>setHovered(false)</code>
<code>onPressed(e, canvas)</code>	判斷落點：在 <code>port</code> → <code>subState=RESIZING</code> ；在物件 → <code>subState=DRAGGING_OBJECT</code> （記錄 <code>moveBefore</code> 快照）；在空白 → <code>subState=RUBBER_BANDING</code> （取消所有選取）
<code>onDragged(e, canvas)</code>	<code>RESIZING</code> ：呼叫 <code>applyResize</code> 即時更新物件大小； <code>DRAGGING_OBJECT</code> ：呼叫 <code>object.moveTo</code> 即時移動； <code>RUBBER_BANDING</code> ：更新橡皮筋矩形
<code>onReleased(e, canvas)</code>	<code>RESIZING</code> ：快照 <code>boundsAfter</code> → <code>pushHistory(ResizeObjectCommand)</code> ； <code>DRAGGING_OBJECT</code> ：快照 <code>moveAfter</code> → <code>pushHistory(MoveObjectsCommand)</code> ； <code>RUBBER_BANDING</code> ：框選完全落於矩形內的物件
<code>onClicked(e, canvas)</code>	找出點選物件 → <code>unselect</code> 其他物件 → <code>select</code> 此物件 → 調深度為最小

方法	行為
<code>applyResize(newPoint)</code>	根據 <code>resizePort</code> 和 <code>ResizeConstraint</code> ，計算新的 <code>x/y/width/height</code> （含最小尺寸限制與交叉方向處理），呼叫 <code>setBounds</code>

CreateObjectStrategy（建立形狀策略）

方法	行為
<code>onReleased(e, canvas)</code>	確認滑鼠在 <code>Canvas</code> 範圍內 → 建立 <code>RectObject</code> 或 <code>OvalObject</code> → 包成 <code>CreateObjectCommand</code> → <code>canvas.execute</code>
（呼叫後）	結束後呼叫 <code>ModeManager.restorePreviousMode()</code> 回到原模式， <code>ButtonPanel</code> 收到通知後高亮更新

CreateLinkStrategy（建立連線策略）

方法	行為
<code>onPressed(e, canvas)</code>	找最近 <code>port</code> 的 <code>BasicObject</code> → 記錄 <code>sourceObject</code> 和 <code>sourcePortIndex</code> → <code>subState=LINKING</code>
<code>onDragged(e, canvas)</code>	更新 <code>tempEnd</code> 座標 → <code>repaint</code> （畫虛線預覽）
<code>onReleased(e, canvas)</code>	找 <code>targetObject</code> 和 <code>targetPortIndex</code> → 驗證不同物件 → <code>buildLink()</code> → <code>CreateLinkCommand</code> → <code>execute</code>
<code>buildLink(src, srcIdx, tgt, tgtIdx)</code>	工廠方法，依目前 <code>EditorMode</code> 建立 <code>AssociationLink</code> / <code>GeneralizationLink</code> / <code>CompositionLink</code>

BasicObject（抽象形狀）

方法	行為
<code>getPorts()</code>	若 <code>portsCache</code> 為 <code>null</code> 則呼叫 <code>computePorts()</code> 重算；否則直接回傳快取
<code>computePorts()</code>	子類實作，計算每個 <code>port</code> 的絕對座標（ <code>RectObject</code> =8個， <code>OvalObject</code> =4個）
<code>getNearestPortIndex(pt)</code>	用 <code>Chebyshev</code> 距離找最近的 <code>port</code> ，回傳其 <code>index</code>
<code>getResizeConstraint(portIdx)</code>	回傳此 <code>port</code> 的縮放軸鎖（邊中點鎖單軸，角點自由縮放）
<code>getResizeAnchor(portIdx)</code>	回傳縮放時的對側固定錨點座標
<code>setBounds(x, y, w, h)</code>	更新座標與尺寸，並將 <code>portsCache</code> 設為 <code>null</code> 使其失效
<code>move(dx, dy) / moveTo(x, y)</code>	移動物件並清空 <code>portsCache</code> ，確保 <code>LinkObject.draw()</code> 下次取到最新 <code>port</code> 座標
<code>draw(g)</code>	Template Method：依序呼叫 <code>drawShape</code> → <code>drawPorts</code> （ <code>hoverd/selected</code> 才顯示）→ <code>drawLabel</code>

CompositeObject (群組物件)

方法	行為
<code>addChild(obj)</code>	將子物件加入 children 列表
<code>removeChild(obj)</code>	從 children 列表移除子物件
<code>getBounds()</code>	遞迴計算所有子物件的最小外包矩形
<code>draw(g)</code>	遞迴 draw 所有 children；若 selected 則繪製虛線外框
<code>move(dx, dy)</code>	遞迴對所有 children 呼叫 move
<code>contains(x, y)</code>	以 getBounds() 的外包矩形做命中測試

Command 子類

Command	redo()	undo()
<code>CreateObjectCommand</code>	<code>rawAddObject(obj)</code>	<code>rawRemoveObject(obj)</code>
<code>CreateLinkCommand</code>	<code>rawAddLink(link)</code>	<code>rawRemoveLink(link)</code>
<code>MoveObjectsCommand</code>	套用 <code>moveAfter Map</code> 中每個物件的 <code>moveTo</code>	套用 <code>moveBefore Map</code> 復原
<code>ResizeObjectCommand</code>	套用 <code>boundsAfter</code> 呼叫 <code>setBounds</code>	套用 <code>boundsBefore</code> 復原
<code>GroupCommand</code>	移除 children → 建立 CompositeObject → <code>rawAddObject</code>	<code>rawRemoveObject(composite)</code> → 還原 children
<code>UngroupCommand</code>	<code>rawRemoveObject(composite)</code> → <code>rawAddObject</code> 每個 child	復原為 CompositeObject
<code>SetLabelCommand</code>	設定新的 <code>labelName</code> + <code>labelColor</code>	還原舊的 <code>labelName</code> + <code>labelColor</code>

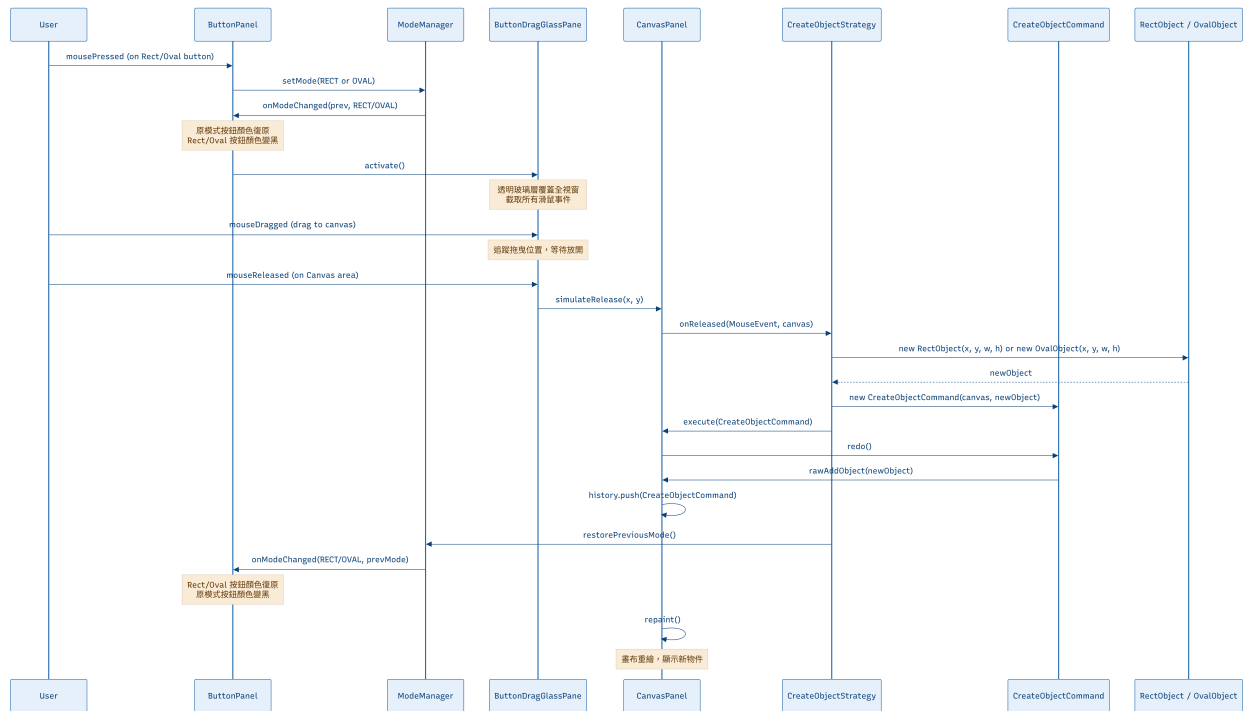
ModeManager (狀態管理)

方法	行為
<code>setMode(mode)</code>	儲存 <code>previousMode</code> 、更新 <code>currentMode</code> 、通知所有 <code>ModeChangeListener</code>
<code>restorePreviousMode()</code>	將 <code>currentMode</code> 恢復為 <code>previousMode</code> ，通知 listeners（用於建立物件後回復模式）
<code>addListener(l)</code>	註冊 <code>ModeChangeListener</code> （ <code>ButtonPanel</code> 實作此介面）

三、把每個 Use Case 都用 Sequence Diagram 表示出來

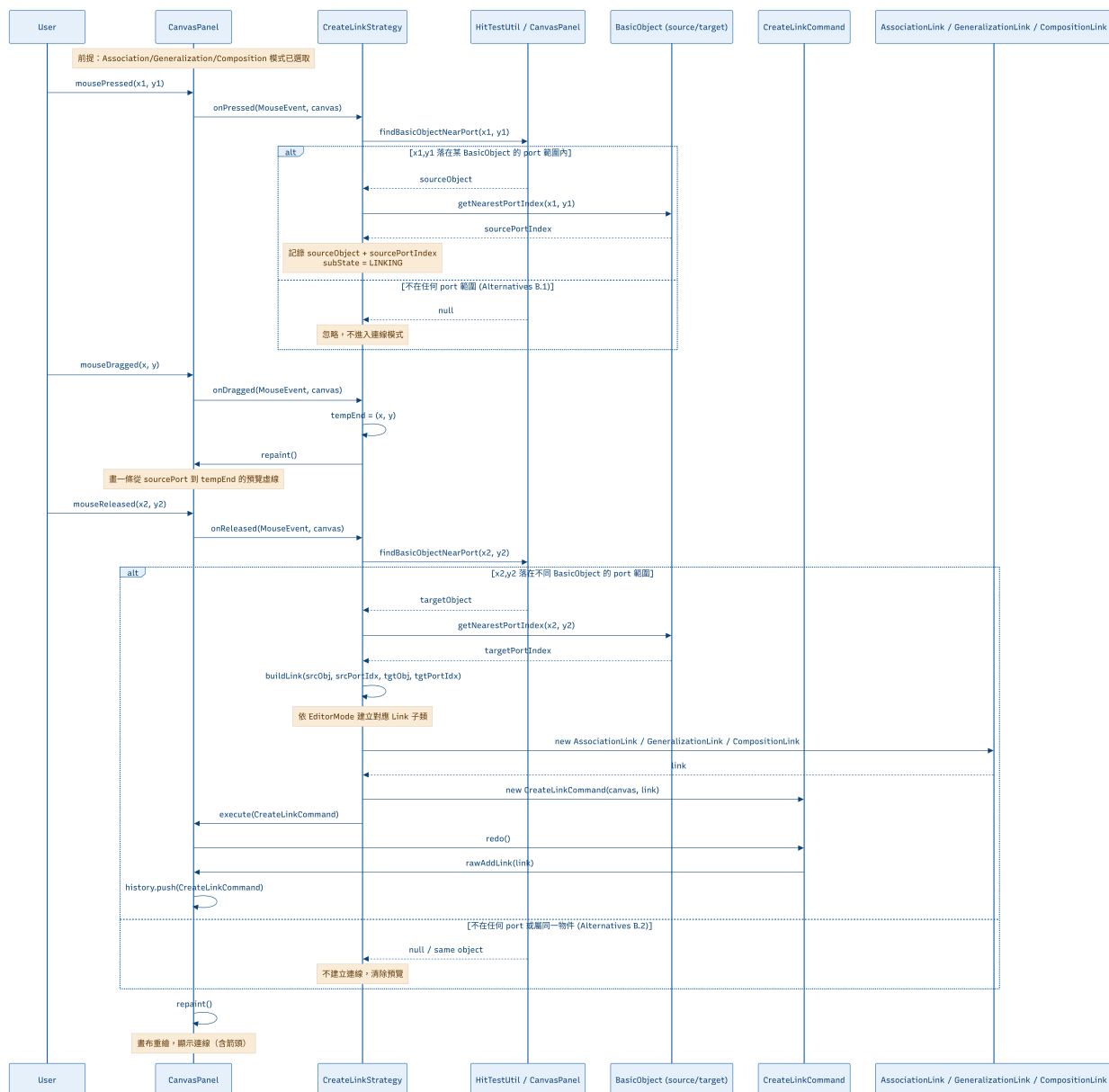
- USE CASE-A：Creating an Object（建立形狀）

思路：User 在工具按鈕上按下後拖曳至 Canvas 放開，`ButtonDragGlassPane` 作為中介接管全視窗事件，模擬在 Canvas 上放開的效果。



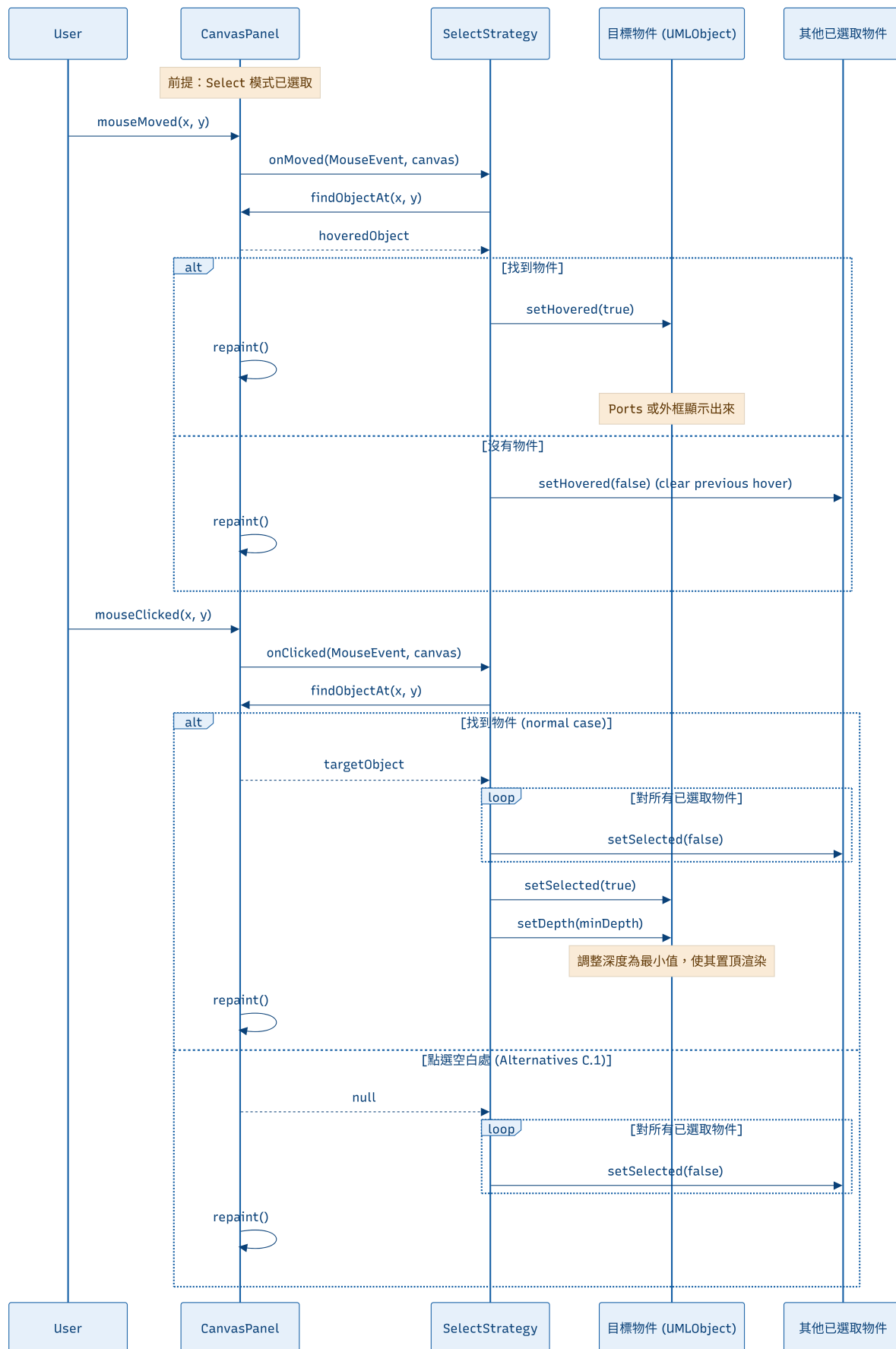
- USE CASE-B：Creating a Link（建立連線）

思路：Link 模式下，onPressed 找 source port，onDragged 畫預覽線，onReleased 找 target port 後由 buildLink 工廠建立對應連線物件。



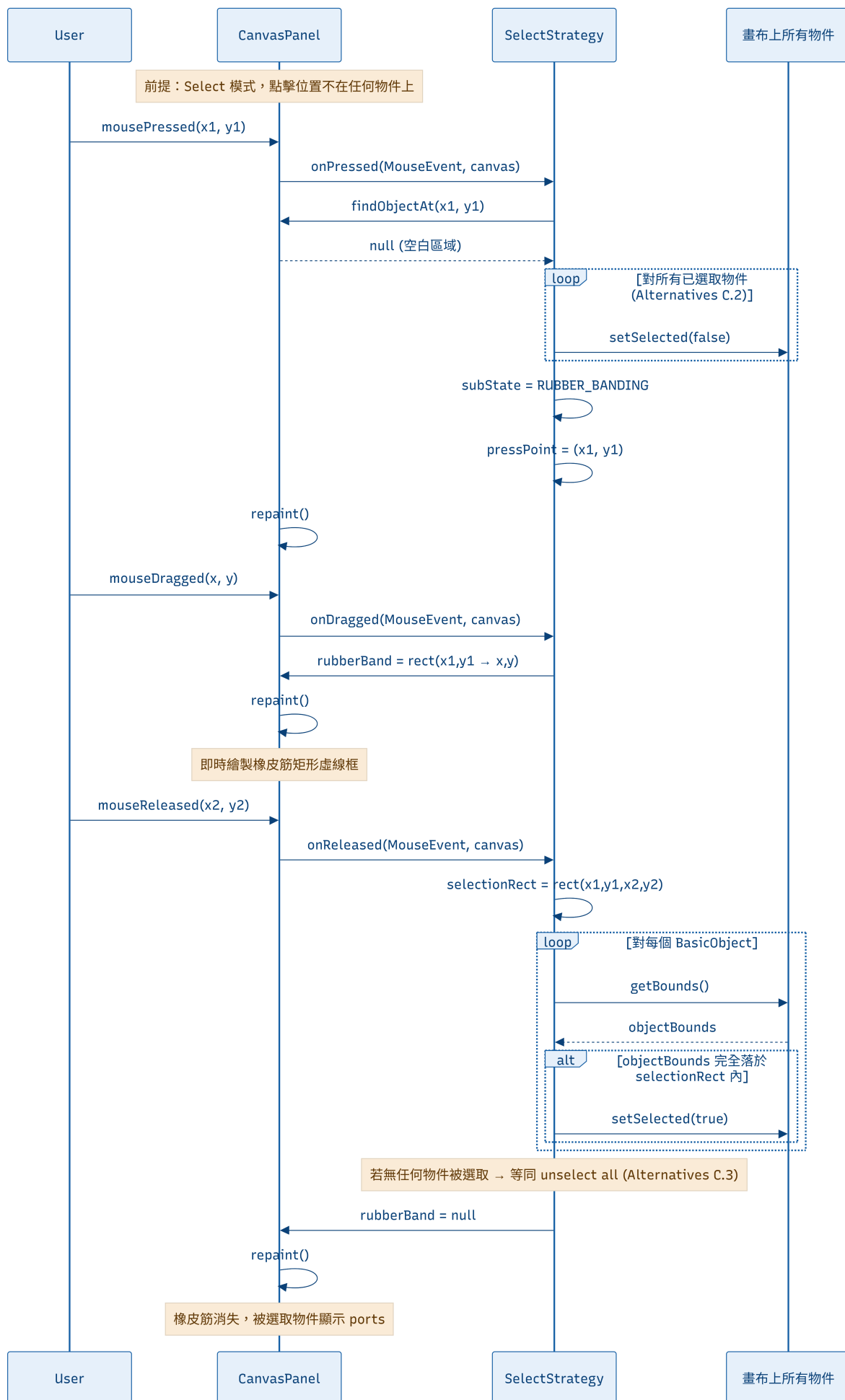
- USE CASE-C (Case 1)：Select / Unselect — 單擊選取

思路：onMoved 做 hover 偵測顯示 ports，onClicked 做正式選取並取消其他物件選取狀態，調整 depth 至最小使其置頂。



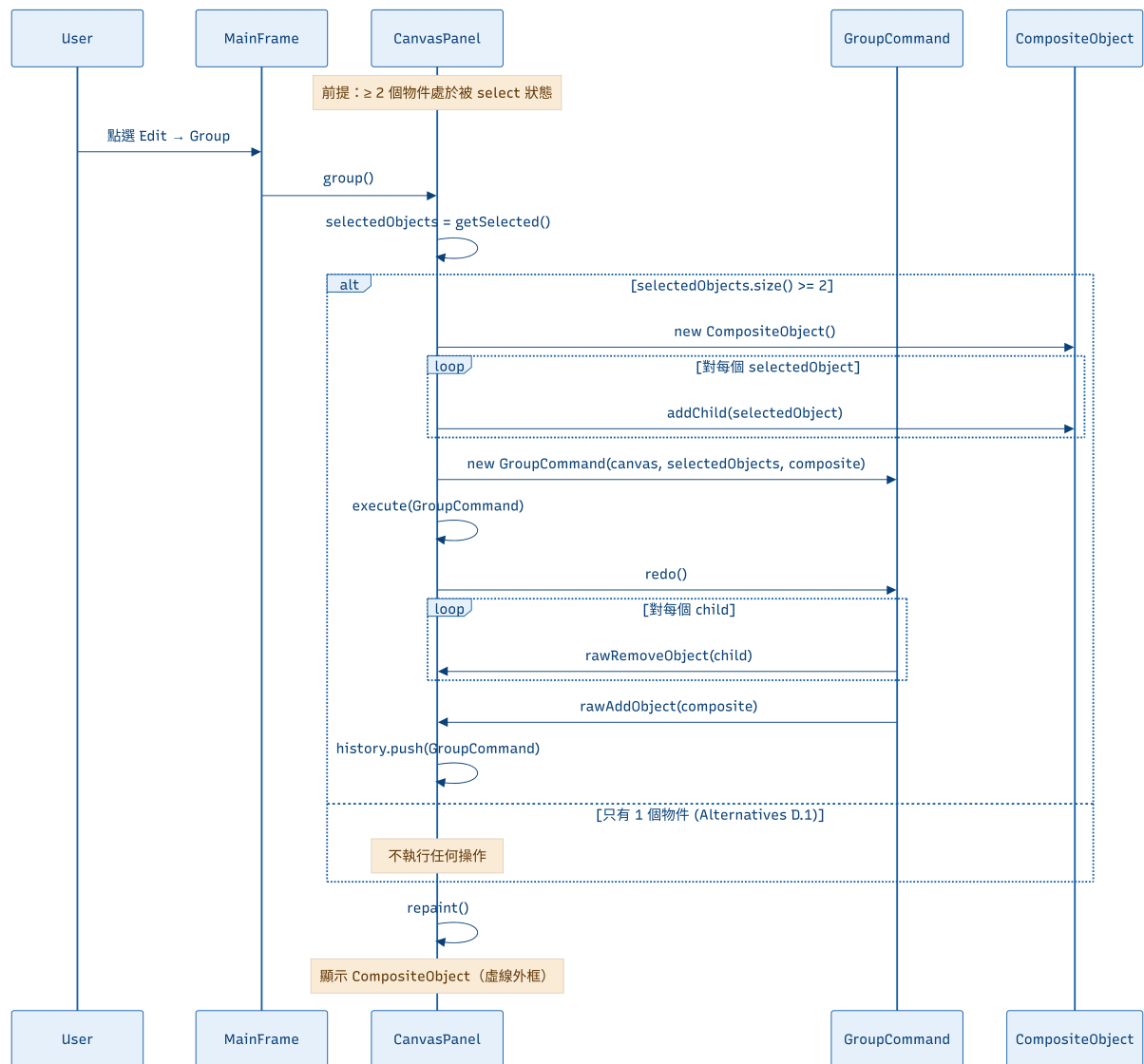
- USE CASE-C (Case 2)：Select / Unselect — 框選（Rubber Band）

思路：在空白處按下並拖曳，即時顯示橡皮筋矩形；放開後計算哪些物件完全落於矩形內，選取它們。



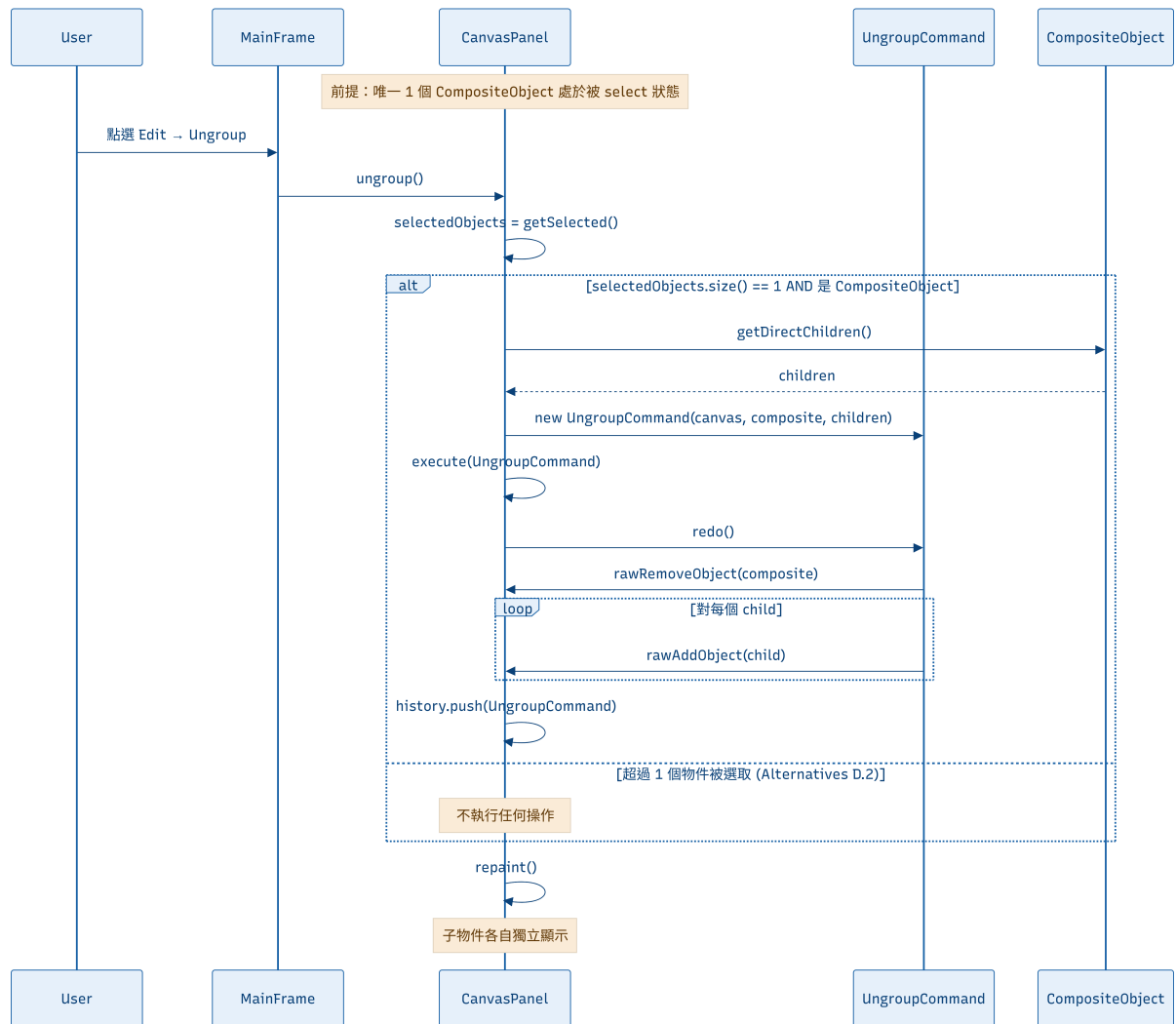
- USE CASE-D (Case 1) : Group Objects (群組)

思路：確認 ≥ 2 個物件被選取後，透過 GroupCommand 把它們包成一個 CompositeObject，原物件從頂層列表移除。



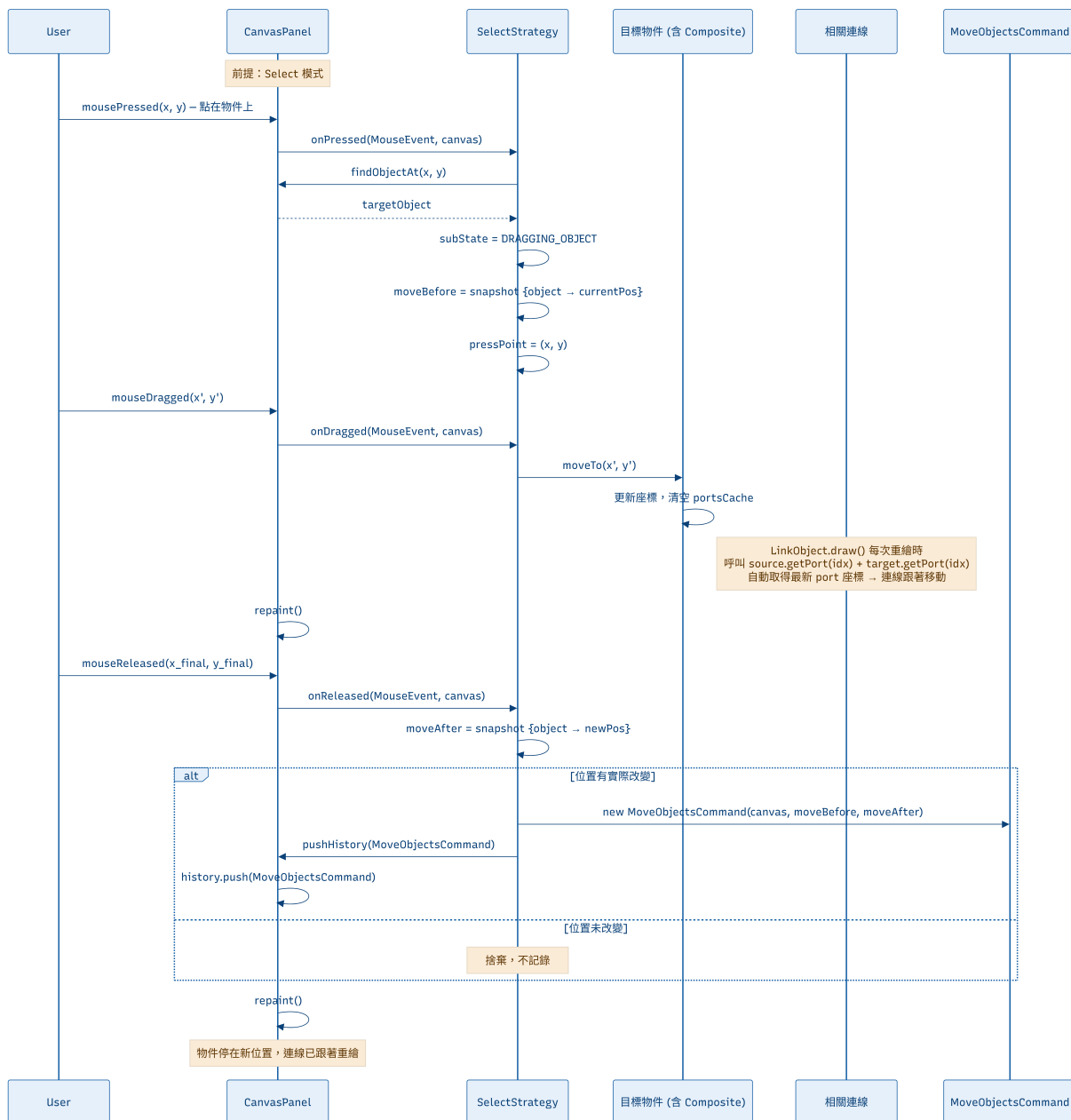
- USE CASE-D (Case 2) : Ungroup Objects (解群組)

思路：確認唯一選取物件是 CompositeObject 後，透過 UngroupCommand 將其拆解，子物件回到頂層列表。



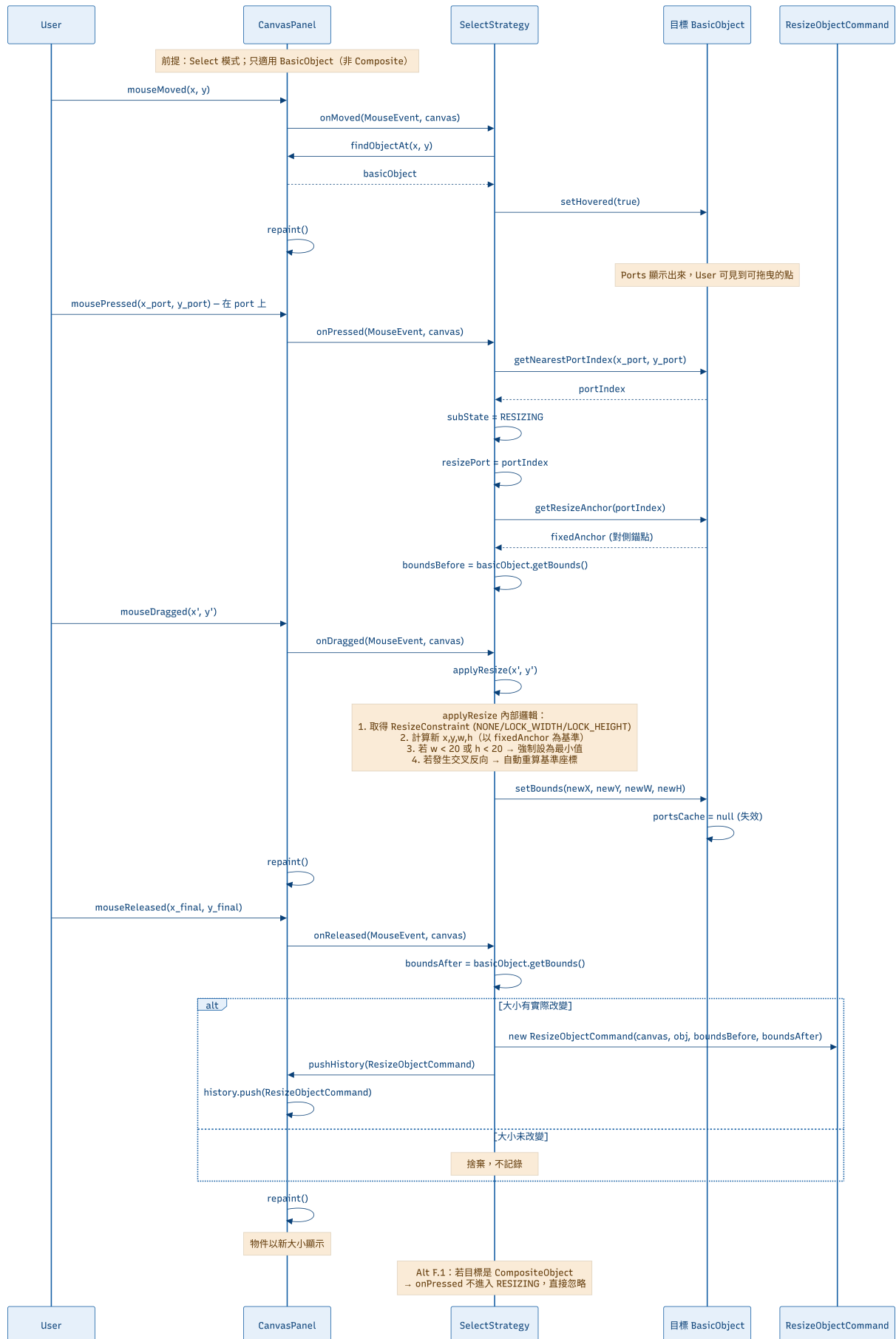
- USE CASE-E：Move Objects（移動物件）

思路：onPressed 快照 moveBefore，onDragged 即時移動（連線 port 動態追蹤自動更新），onReleased 快照 moveAfter 後 pushHistory。



- USE CASE-F：Resize Objects（調整物件大小）

思路：在 BasicObject 的 port 上按下後進入 RESIZING 子狀態，onDragged 即時 applyResize（含軸鎖定、最小尺寸、交叉反向處理），onReleased pushHistory。



• USE CASE-G：Customize Label Style（自定義標籤）

思路：選取 BasicObject 後從 Edit Menu 開啟 LabelDialog；OK 時包成 SetLabelCommand 執行；Cancel 則不做任何改變。

